



ΣΧΟΛΗ ΨΗΦΙΑΚΗΣ ΤΕΧΝΟΛΟΓΙΑΣ ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ
ΤΗΛΕΜΑΤΙΚΗΣ

ΑΚΑΔ. ΕΤΟΣ 2022-2023

ΑΘΗΝΑ 16- 12 - 2022

2^η ΟΜΑΔΑ ΑΣΚΗΣΕΩΝ

ΓΙΑ ΤΟ ΜΑΘΗΜΑ

"Θέματα Σχεδίασης Βάσεων Δεδομένων"

(Δημησιάνου Κερκύρα-it22026

Παπαδόπουλος Γιάννης-it22085)

A-ΕΡΩΤΗΜΑΤΑ

1^ο μέρος-Εντολές Δημιουργίας Ευρετηρίων

CREATE **UNIQUE/BITMAP** INDEX ON **CLUSTER_INDEX_CLAUSE**
ON TABLE_INDEX_CLAUSE

Αν δεν προσδιορίσουμε τον τύπο του ευρετηρίου η Oracle δημιουργεί ευρετήρια που βασίζονται σε B+ δέντρα.

UNIQUE: Χρησιμοποιείται για να δείξει ότι η στήλη/οι στήλες πάνω στις οποίες γίνεται η ευρετηριοποίηση είναι μοναδικές.(όπως π.χ. το ID κάθε EMPLOYEE που είναι PRIMARY KEY και είναι μοναδικό).

BITMAP: Χρησιμοποιείται για να δείξει ότι το ευρετήριο βασίζεται σε bitmap λογική. Κάθε πλειάδα στο αρχείο αντιπροσωπεύεται από ένα rowid. Το bitmap ευρετήριο αποθηκεύει για κάθε rowid ένα ζεύγος κλειδί-τιμή ως bitmap. Αν το bit είναι 1, τότε σημαίνει ότι η πλειάδα με το συγκεκριμένο rowid περιέχει αυτό το ζεύγος κλειδί-τιμή. Το bitmap ευρετήριο είναι χρήσιμο για πεδία με λίγες διαφορετικές τιμές.

BITMAP JOIN INDEX: Το Bitmap Join Index επεκτείνει αυτήν την έννοια έτσι ώστε το INDEX να περιέχει τα δεδομένα για την υποστήριξη του JOIN, επιτρέποντας στο ερώτημα να ανακτήσει τα δεδομένα από το INDEX αντί να παραπέμπει στα JOINED TABLES. Εφόσον οι πληροφορίες συμπίπτουν σε ένα bitmap, το μέγεθος της δομής που προκύπτει είναι σημαντικά μικρότερο από την αντίστοιχη υλοποιημένη προβολή

CLUSTER_INDEX_CLAUSE:

- CLUSTER / SCHEMA.CLUSTER / INDEX_ATTRIBUTES

Δείχνει το πεδίο διάταξης (δεν είναι κλειδί) πάνω στο οποίο θα γίνει η ευρετηριοποίηση. Το ευρετήριο συστάδων περιέχει μία εγγραφή για κάθε διακεκριμένη τιμή του πεδίου διάταξης η οποία αποτελείται από την τιμή αυτή και ένα δείκτη στο block του αρχείου που περιέχει μία εγγραφή με την τιμή αυτή στο πεδίο συστάδας.

CLUSTER: αναφέρεται στο πεδίο συστάδας

SCHEMA.CLUSTER: SCHEMA-αναφέρεται σε μία συλλογή βάσεων δεδομένων που αποτελείται από tables, triggers, stored procedures etc. Εδώ το .CLUSTER είναι ένας νέος πίνακας στο SCHEMA τύπου CLUSTER όπου αποθηκεύει δεδομένα πινάκων από το αρχείο και το ευρετήριο συστάδων.

INDEX_ATTRIBUTES(optional)

TABLE_INDEX_CLAUSE:

- SCHEMA.TABLE/ T_ALIAS/ INDEX_EXPR/ ASC | DESC/ INDEX_PROPERTIES

SCHEMA.TABLE: TABLE- είναι ένα structure με πολλαπλές πλειάδες, καθεμία από τις οποίες έχει τα χαρακτηριστικά που ορίζονται από το SCHEMA.

T_ALIAS: χρησιμοποιείται για να καθορίσουμε ένα όνομα συσχέτισης για τον πίνακα στον οποίο δημιουργείται το INDEX.

INDEX_EXPR: καθορίζουμε την στήλη ή την έκφραση στήλης στην οποία βασίζεται το INDEX.

ASC | DESC: χρησιμοποιείται για να υποδείξουμε αν το INDEX πρέπει να δημιουργηθεί με αύξουσα ή φθίνουσα σειρά.

Extra resources:

<https://www.geeksforgeeks.org/create-schema-in-sql-server/>

<https://www.geeksforgeeks.org/sql-clusters/>

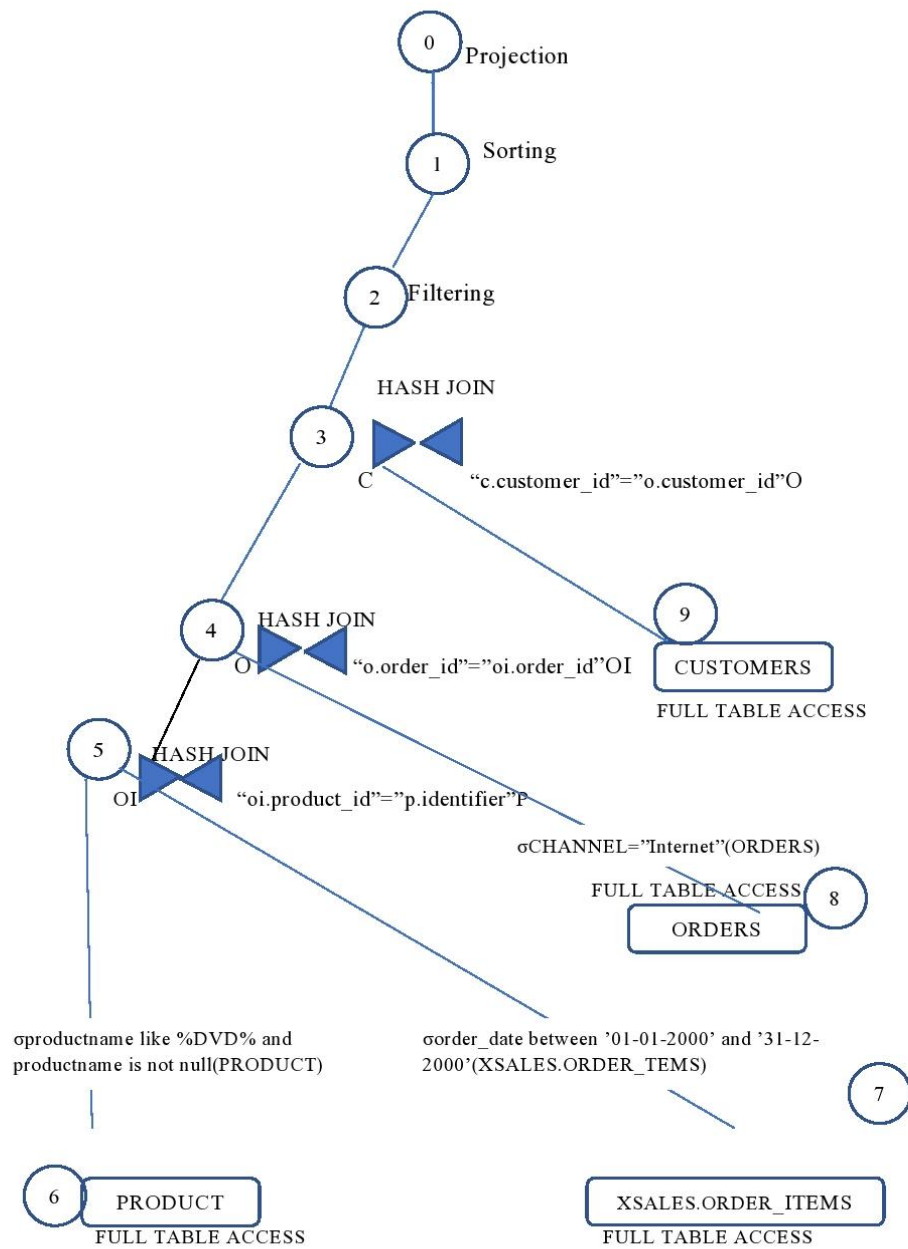
https://docs.oracle.com/cd/B19306_01/server.102/b14200/statements_5010.htm

3^ο μέρος-Βελτιστοποίηση ερωτήματος

Με βάση τον πίνακα PLAN_TABLE παρατηρούμε τις εξής τιμές:

- 1) COST:4373 blocks CPU_COST:920583402 blocks IO_COST:4350 blocks. Η πιο χρονοβόρα ενέργεια μετά το select statement είναι το hash join μεταξύ customers και orders που επιλέγει να κάνει ο optimizer στα πεδία c.customer_id=o.customer_id με κόστος αναζήτησης:4373 blocks.
- 2) Σύνολο εκτιμώμενων επιστρεφόμενων πλειάδων:678 = συνολικές τελικές πλειάδες

ΠΛΑΝΟ ΕΚΤΕΛΕΣΗΣ



3) Για την βελτιστοποίηση του ερωτήματος αποφασίζουμε να χρησιμοποιήσουμε τα εξής ευρετήρια:

```
CREATE INDEX CINDEXT ON CUSTOMERS(NAME,CUSTOMER_ID)
```

```
CREATE INDEX PRODUCTINDEX ON PRODUCT(PRODUCTNAME,IDENTIFIER)
```

```
CREATE INDEX ORDERINDEX ON ORDERS(ORDER_ID,CHANNEL)
```

Δημιουργούμε δηλαδή 3 default(based on B+ tree structure) indexes πάνω στους πίνακες CUSTOMER,PRODUCT,ORDERS στα πεδία (NAME,CUSTOMER_ID),(PRODUCTNAME,IDENTIFIER),(ORDER_ID,CHANNEL) αντίστοιχα.

Τρέχοντας το πλάνο εκτέλεσης μετά την προσθήκη των ευρετηρίων παρατηρούμε ότι παρά το γεγονός ότι το βάθος της αναζήτησης αυξάνεται το κόστος μειώνεται στα 3322 blocks (67 block για αναζήτηση συνολικά στα ευρετήρια και 3255 block για αναζήτηση στο αρχείο).

4ο μέρος-Θεωρητικός υπολογισμός κόστους

//βρίσκει για κάθε σχέση μέγεθος blocks και bytes
SELECT * FROM USER_SEGMENTS;

//για να βρω μέγεθος block και bytes για την σχέση xsales.orders
SELECT * FROM DBA_SEGMENTS
WHERE OWNER='XSALES';

Άρα βρίσκουμε:

<u>RELATIONSHIP</u>	<u>BLOCKS</u>	<u>INITIAL BYTES</u>	<u>INITIAL TUPLES</u>
Customers	640	5242880	55000
Product	8	65536	72
Order	6912	56623104	916039
Order_items	8704	71303168	916039

Σύμφωνα με το πλάνο που επέλεξε ο optimizer το κόστος για κάθε βήμα είναι το εξής:

1. Table access full Customers: cost $640 * 2(\text{read/write I/O}) = 1280$ blocks
2. Table access full Orders:
 - a. 6912 blocks (read)
 - b. $S_f = 6912$ blocks
 - c. $916039 / 6912 = 132$ tuples per block
 - d. (where channel='Internet') $\rightarrow 117686$ tuples stored in $117686 / 132 = 891$ blocks(write)
 - e. Άρα cost = $6912 + 891 = 7803$ blocks
3. Table access full Order_items:
 - a. cost 8704(read)
 - b. $S_f = 8704$ blocks
 - c. $916039 / 8704 = 105$ tuples per block
 - d. (where order_date >='01-01-2000' and order_date <='31-12-2000') $\rightarrow 231916$ tuples stored in $231916 / 105 = 2209$ blocks(write)
 - e. Άρα cost = $8704 + 2209 = 10913$ blocks
4. Table access full Products:
 - a. 8 blocks(read)
 - b. $S_f = 8$ blocks
 - c. $72 / 8 = 9$ tuples per blocks
 - d. (where product_name like %DVD%) $\rightarrow 6$ tuples stored in $6 / 9 = 0,25$ block(write)
 - e. Άρα cost = $8 + 0,25 \sim 8$ blocks
5. Hash join "oi.product_id"="p.identifier"
 - a. Partitioning: $2 * (b_{oi} + b_p) = 2 * (2209 + 0,25)$
 - b. Matching: $(b_{oi} + b_p) = (2209 + 0,25)$
 - c. Άρα cost: $3 * (b_{oi} + b_p) = 3 * (2209) = 6627$ blocks
 - d. Returns 916039 tuples
 - e. $916039 / 2209 = 415$ tuples/block της order_items
 - f. $2209 / 415 = 5$ blocks της order_items για να αποθηκευτούν οι 916039 tuples.
 - g. Άρα μένουν $2209 - 5 = 2204$ blocks
6. Hash join "o.order_id"="oi.order_id"
 - a. Partitioning: $2 * (b_o + b_{oi}) = (2204 + 6912)$
 - b. Matching: $(b_o + b_{oi})$
 - c. Άρα cost: $3 * (b_o + b_{oi}) = 27348$
 - d. Returns 76921593 tuples
 - e. $76921593 / 6912 = 11129$ tuples/block της orders
 - f. $6912 / 11129 = 0,65$ blocks της orders για να αποθηκευτούν οι 76921593 tuples

g. Άρα $6912 - 0,65 = 6911,5 \sim 6911$ blocks

7. Hash join “c.customer_id”=”o.customer_id”

a. Partitioning: $2 * (bc + bo) = 2 * (6911 + 640)$

b. Matching: $(bc + bo)$

c. Άρα cost: $3 * (bc + bo) = 3 * (6911 + 640) = 7551$

d. Returns 76921593 tuples

e. $76921593 / (6911 + 640) = 7551$ blocks χρειάζονται για να αποθηκευτούν οι τελικές πλειάδες

8. Filtering: read 7551 blocks άρα cost: 7551 blocks

9. Sorting: cost: 7551 blocks

10. Projection: cost: 7551 blocks

Τελικό κόστος 7551 blocks. Σε σχέση με τον optimizer παρατηρούμε ότι το θεωρητικό αποτέλεσμα διαφέρει κατά 3000 blocks. Ο optimizer υπολογίζει με ακρίβεια το κόστος κάθε λειτουργίας που εκτελείται ενώ σε θεωρητική ανάλυση βρίσκεται προσεγγιστικά.

Sql script

```
ALTER SESSION SET NLS_DATE_FORMAT='DD-MM-YYYY';

--1o/creating the tables
--relationship: CUSTOMERS
CREATE TABLE CUSTOMERS AS
SELECT NAME, ID AS
CUSTOMER_ID, GENDER, MARITAL_STATUS, INCOME_LEVEL, BIRTH_DATE
FROM XSALES.CUSTOMERS;

--add column age
ALTER TABLE CUSTOMERS
ADD AGE NUMBER(20);

--find age for every customer
UPDATE CUSTOMERS
SET AGE=(SELECT
EXTRACT(YEAR FROM sysdate) - EXTRACT(YEAR FROM to_date(BIRTH_DATE,
'DD-MM-YYYY'))
FROM dual);
```

--add column age_group

```
ALTER TABLE CUSTOMERS  
ADD AGE_GROUP VARCHAR(30);
```

--update age_group column

```
UPDATE CUSTOMERS  
SET AGE_GROUP='under 30'  
WHERE AGE<=30;
```

```
UPDATE CUSTOMERS  
SET AGE_GROUP='30-40'  
WHERE AGE>30 AND AGE<=40;
```

```
UPDATE CUSTOMERS  
SET AGE_GROUP='40-50'  
WHERE AGE>40 AND AGE<=50;
```

```
UPDATE CUSTOMERS  
SET AGE_GROUP='50-60'  
WHERE AGE>50 AND AGE<=60;
```

```
UPDATE CUSTOMERS  
SET AGE_GROUP='60-70'  
WHERE AGE>60 AND AGE<=70;
```

```
UPDATE CUSTOMERS  
SET AGE_GROUP='above 70'  
WHERE AGE>70;
```

--drop column age

```
ALTER TABLE CUSTOMERS  
DROP COLUMN AGE;
```

--find all the different values of column income level

--and sort them alphabetically

```
SELECT INCOME_LEVEL  
FROM CUSTOMERS  
GROUP BY INCOME_LEVEL  
ORDER BY INCOME_LEVEL;
```

--update income level

```
UPDATE CUSTOMERS  
SET INCOME_LEVEL='high'  
WHERE INCOME_LEVEL='J: 190,000 - 249,999'  
OR INCOME_LEVEL='K: 250,000 - 299,999' OR INCOME_LEVEL='L: 300,000 and  
above';
```

```
UPDATE CUSTOMERS
SET INCOME_LEVEL='medium'
WHERE INCOME_LEVEL='F: 110,000 - 129,999' OR INCOME_LEVEL='G: 130,000 - 149,999'
      OR INCOME_LEVEL='H: 150,000 - 169,999' OR INCOME_LEVEL='I: 170,000 - 189,999';
```

```
UPDATE CUSTOMERS
SET INCOME_LEVEL='low'
WHERE INCOME_LEVEL='A: Below 30,000' OR INCOME_LEVEL='B: 30,000 - 49,999'
      OR INCOME_LEVEL='C: 50,000 - 69,999' OR INCOME_LEVEL='D: 70,000 - 89,999'
OR
      INCOME_LEVEL='E: 90,000 - 109,999';
```

```
UPDATE CUSTOMERS
SET INCOME_LEVEL='unknown'
WHERE INCOME_LEVEL IS NULL;
```

--find all the different values of column marital_status
--and sort them alphabetically

```
SELECT MARITAL_STATUS
FROM CUSTOMERS
GROUP BY MARITAL_STATUS
ORDER BY MARITAL_STATUS;
```

--update column marital_status

```
UPDATE CUSTOMERS
SET MARITAL_STATUS='married'
WHERE MARITAL_STATUS='Married' OR MARITAL_STATUS='Mabsent'
      OR MARITAL_STATUS='married' OR MARITAL_STATUS='Mar-AF';
```

```
UPDATE CUSTOMERS
SET MARITAL_STATUS='single'
WHERE MARITAL_STATUS='Divorc.' OR MARITAL_STATUS='NeverM'
      OR MARITAL_STATUS='Separ.' OR MARITAL_STATUS='Widowed' OR
      MARITAL_STATUS='divorced' OR MARITAL_STATUS='widow' OR
      MARITAL_STATUS='single';
```

```
UPDATE CUSTOMERS
SET MARITAL_STATUS='unknown'
WHERE MARITAL_STATUS IS NULL;
```

```
DESC CUSTOMERS;
SELECT * FROM CUSTOMERS;
```

--relationship:PRODUCTS

--create table product from xsales

```
CREATE TABLE PRODUCTS AS
SELECT *
FROM XSALES.PRODUCTS;
```

--create table categories from xsales

```
CREATE TABLE CATEGORIES AS
SELECT *
FROM XSALES.CATEGORIES;
```

--add constraints to table products and categories in

--order to do the join

```
ALTER TABLE CATEGORIES
MODIFY ID PRIMARY KEY;
```

```
ALTER TABLE PRODUCTS
```

```
ADD CONSTRAINT fk1 FOREIGN KEY(SUBCATEGORY_REFERENCE) REFERENCES
CATEGORIES (ID);
```

--create relationship product

```
CREATE TABLE product AS
SELECT p.IDENTIFIER,p.NAME AS PRODUCTNAME,c.NAME AS
CATEGORYNAME,p.LIST_PRICE
FROM CATEGORIES c JOIN PRODUCTS p ON c.ID=p.SUBCATEGORY_REFERENCE;
```

```
DESC product;
```

```
SELECT * FROM product;
```

--drop constraints in order to drop unused tables

```
ALTER TABLE PRODUCTS
DROP CONSTRAINT fk1;
```

--relationship:ORDERS

```
CREATE TABLE ORDERS AS
SELECT OI.ORDER_ID,OI.PRODUCT_ID,O.CUSTOMER_ID,OI.AMOUNT AS
PRICE,OI.COST,O.CHANNEL,OI.ORDER_DATE,O.ORDER_FINISHED
FROM XSALES.ORDERS O JOIN XSALES.ORDER_ITEMS OI ON O.ID=OI.ORDER_ID;
```

```
ALTER TABLE ORDERS
```

```
ADD DAYS_TO_PROCESS NUMBER(20);
```

--update column days_tos_process

```
UPDATE ORDERS
SET DAYS_TO_PROCESS=ABS(ORDER_FINISHED-ORDER_DATE);
```

--drop unused columns

```
ALTER TABLE ORDERS  
DROP COLUMN ORDER_FINISHED;
```

```
ALTER TABLE ORDERS  
DROP COLUMN ORDER_DATE;
```

```
DESC ORDERS;  
SELECT * FROM ORDERS;
```

--3o/optimizer's analysis

```
EXPLAIN PLAN FOR  
SELECT c.name, o.price, oi.quantity, p.productname  
FROM customers c JOIN orders o ON c.customer_id=o.customer_id  
      JOIN XSALES.order_items oi ON o.order_id=oi.order_id  
      JOIN product p ON oi.product_id=p.identifier  
WHERE p.productname LIKE '%DVD%'  
      AND oi.order_date BETWEEN '01-01-2000' AND '31-12-2000'  
      AND o.channel='Internet'  
ORDER BY c.name;
```

--view optimizer's analysis

```
SELECT * FROM plan_table  
START WITH id=0  
CONNECT BY PRIOR id=parent_id;
```

--optimizing the query

```
CREATE INDEX CINDEX ON CUSTOMERS(NAME,CUSTOMER_ID);  
  
CREATE INDEX PRODUCTINDEX ON PRODUCT(PRODUCTNAME,IDENTIFIER);  
  
CREATE INDEX ORDERINDEX ON ORDERS(ORDER_ID,CHANNEL);
```

--4o/theoretical evaluation of cost

--find the size of a relationship in blocks and bytes

```
SELECT * FROM USER_SEGMENTS ;
```

--find the size of the relationship XSALES.ORDERS in blocks and bytes

```
SELECT *  
FROM dba_segments  
WHERE owner='XSALES';
```

--in order to find the initial number of rows of every table that needs the query

```
SELECT COUNT(*)
```

```
FROM PRODUCT;
```

```
SELECT COUNT(*)  
FROM ORDERS O  
WHERE O.CHANNEL='Internet';
```

```
SELECT COUNT(*)  
FROM XSALES.ORDER_ITEMS OI  
WHERE oi.order_date BETWEEN '01-01-2000' AND '31-12-2000';
```

```
SELECT COUNT(*)  
FROM PRODUCT P  
WHERE p.productname LIKE '%DVD%';
```

--find the tuples that this join returns

```
SELECT COUNT(*)  
FROM ORDERS O JOIN XSALES.ORDER_ITEMS OI  
ON O.ORDER_ID=OI.ORDER_ID;
```

--find the tuples that this join returns

```
SELECT count(*)  
FROM PRODUCT p JOIN XSALES.ORDER_ITEMS oi  
ON oi.product_id=p.identifier;
```

--find the tuples that this join returns

```
SELECT COUNT(*)  
FROM CUSTOMERS c JOIN ORDERS o  
ON c.customer_id=o.customer_id;
```